

REPORTE DE PRÁCTICA: DISEÑO DE DATAPATH (DPTR)



Materia: Arquitectura de Computadoras

Profesor: Jorge Ernesto López Arce Delgado

Integrantes:

Diego Israel González Sánchez

Derek Gabriel Casillas López

Iván Samuel Ortiz Orellana

Omar Luna Reyes

Fecha: 22/04/2026

INTRODUCCIÓN

En esta actividad se implementó un sistema de procesamiento de datos basado en la arquitectura MIPS, enfocándonos en el flujo de instrucciones dentro de un **Datapath**. El objetivo es comprender cómo se conectan los módulos de hardware para ejecutar operaciones aritméticas y lógicas.

El Proceso de Compilación

La compilación es el proceso de traducción donde un código de alto nivel (como C++ o Python) se convierte en lenguaje de máquina (binario) que el hardware puede entender. Este proceso pasa por varias etapas:

1. **Preprocesado:** Se expanden macros y librerías.
2. **Compilación:** Se traduce el código a lenguaje ensamblador.
3. **Ensamblado:** El ensamblador convierte el código en archivos objeto (binarios).
4. **Enlazado (Linking):** Se unen los archivos objeto para crear el ejecutable final.

Instrucciones Tipo R y SLT

Las instrucciones tipo R (Register) realizan operaciones entre registros. Su formato incluye campos para los registros fuente (rs, rt), el registro destino (rd) y el campo funct que especifica la operación exacta.

- **Instrucción SLT (Set on Less Than):** Es fundamental para la toma de decisiones. Su función es comparar dos valores: si el primer registro es menor que el segundo, el resultado en el registro destino será 1; de lo contrario, será 0. Esto permite implementar estructuras de control como los ciclos if o while en hardware.

Operación Ternaria

La operación ternaria es una forma compacta de tomar una decisión en una sola línea de código, común en lenguajes como C y Verilog. Su sintaxis es condición? valor_si_cierto : valor_si_falso. En nuestro diseño, la utilizamos para implementar multiplexores y para la lógica de la instrucción SLT de forma eficiente.

Resumen del Video de Clase

En la sesión se explicó la interconexión de la **Unidad de Control** con el **Banco de Registros** y la **ALU**. Se hizo énfasis en que el Contador de

Programa (PC) debe avanzar de 4 en 4 para leer correctamente los 32 bits de cada instrucción almacenada en una memoria direccionada por bytes.

OBJETIVO

Objetivo General

Diseñar e implementar un Datapath funcional en Verilog que sea capaz de buscar, decodificar y ejecutar instrucciones de tipo R almacenadas en memoria.

Objetivos Particulares

- Desarrollar una **ALU** capaz de realizar operaciones de suma, resta y comparación (SLT).
 - Implementar un **Banco de Registros** con capacidad de lectura asíncrona y escritura sincrónica.
 - Integrar una **Unidad de Control** que genere las señales necesarias (RegWrite, ALUOp) a partir de los códigos de operación.
-

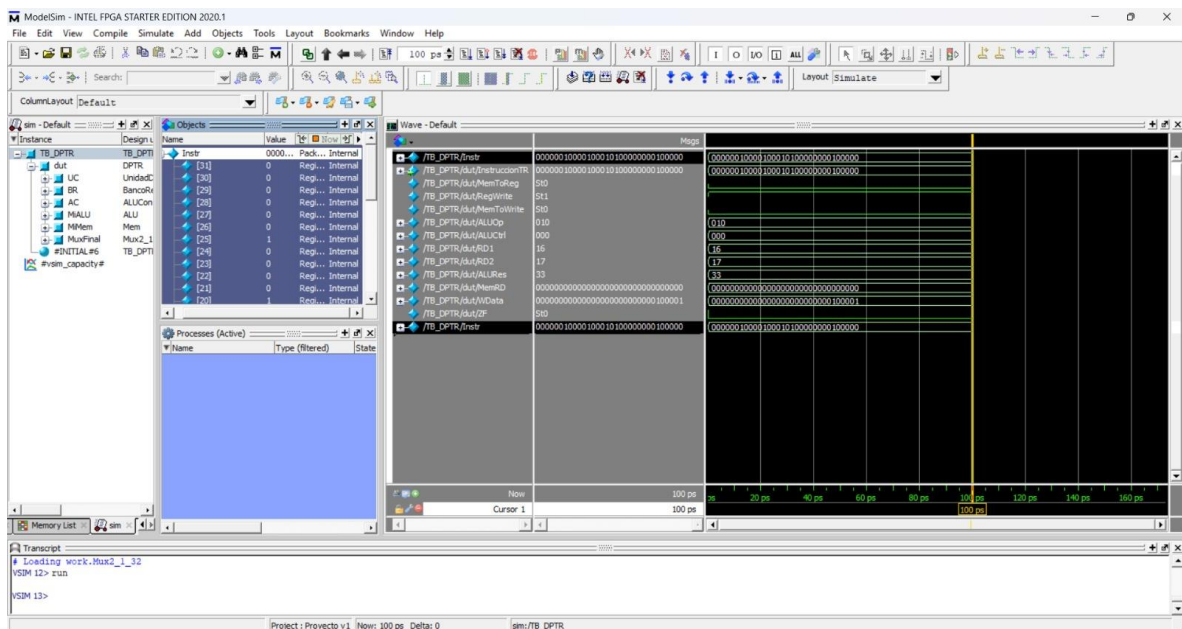
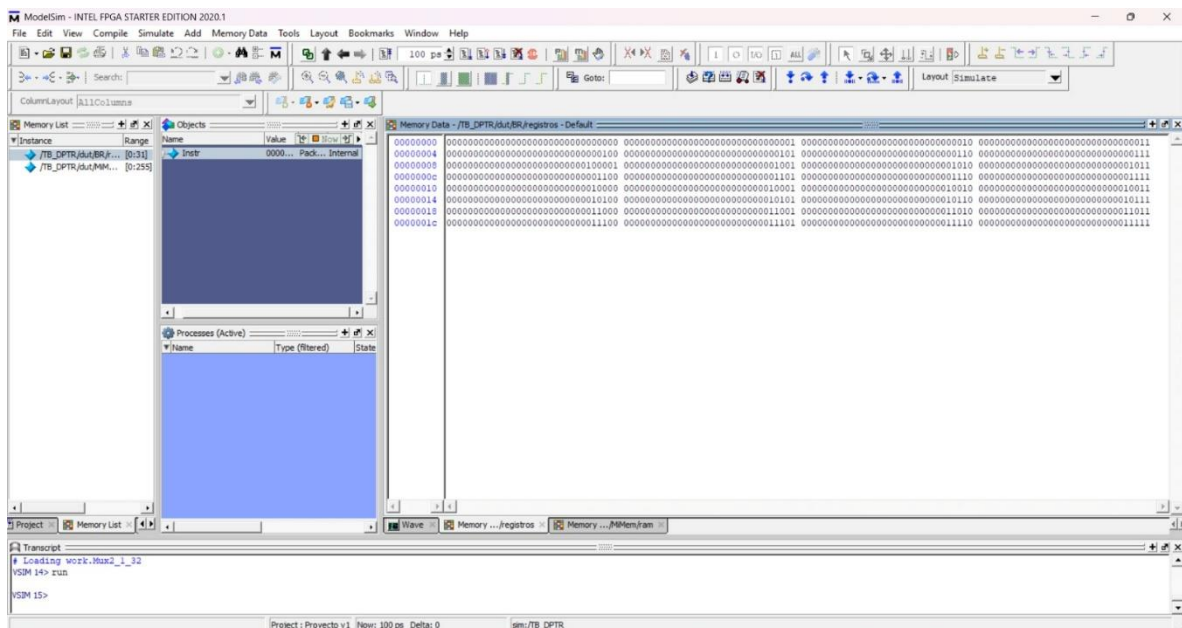
DESARROLLO

El desarrollo se dividió en la creación de módulos modulares que posteriormente fueron integrados en un bloque superior llamado `dptr.v`.

1. **Unidad de Control (`Unidad_Control.v`):** Se programó una estructura case que analiza el opcode de la instrucción. Para el caso `6'b000000` (Tipo R), se activan las señales de escritura en registros y se define el tipo de operación para la ALU.
2. **Banco de Registros (`banco_reg.v`):** Se definieron 32 registros de 32 bits. Implementamos una lógica donde la lectura es inmediata (asíncrona) para acelerar el flujo, mientras que la escritura ocurre únicamente en el flanco de subida del reloj si la señal `we` (write enable) está activa.
3. **Memoria de Instrucciones (`MEM_I.v`):** Dado que la memoria está organizada en bytes (8 bits), configuramos la salida para que concatene cuatro posiciones consecutivas `{MEM[dir], MEM[dir+1], MEM[dir+2], MEM[dir+3]}` para entregar una instrucción completa de 32 bits al sistema.

4. **ALU y Mux:** Se creó la unidad aritmética utilizando la operación ternaria para el manejo de selectores, optimizando el uso de recursos lógicos.
5. **Integración (Conexion.v / dptra.v):** Se unieron los módulos mediante señales tipo wire. El PC (Program Counter) se diseñó para incrementarse en cada ciclo de reloj, permitiendo el avance automático del programa.
6. **Testbench (tb.v):** Se desarrolló un entorno de simulación que genera una señal de reloj y un pulso de reset inicial para limpiar los estados internos antes de comenzar la ejecución de las instrucciones de prueba.

Simulación:



CONCLUSIÓN

Esta práctica fue fundamental para visualizar cómo la teoría de la arquitectura de computadoras se traduce en hardware real. Lo más complejo fue asegurar que el direccionamiento de la memoria coincidiera con el incremento del PC, ya que un error de un solo byte desplaza toda la instrucción. Aprendí que la modularidad en Verilog facilita enormemente la detección de errores (debugging), permitiendo probar la ALU o el Banco de Registros por separado antes de unirlos en el sistema completo.

REFERENCIAS

- Patterson, D. A., & Hennessy, J. L. (2018). *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann.
- Harris, S. L., & Harris, D. (2022). *Digital Design and Computer Architecture*. Morgan Kaufmann.
- IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2005).